

[Paper] GPT-1: Improving Language Understanding by Generative Pre-Training

Posted Sep 10, 2026

By 김민준 (Min-jun, Kim)

Contents >

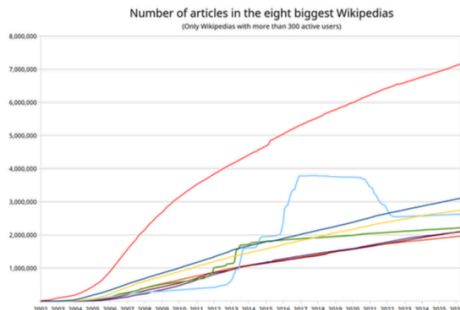
★ 본문 내부의 슬라이드 이미지는 자체 제작한 프레젠테이션을 활용하였습니다.

1. Introduction

Background – 왜 Pre-training이 필요한가?

논문이 제기한 문제: “Labeled Data는 제한적, Unlabeled Data는 압도적으로 풍부하다.”

Unlabeled dataset (2026 기준)



Labeled datasets

STS Benchmark: 8,628 sentences
Quora Question Pairs: 404,290 question pairs
CoLA: 10,657 sentences

document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large

GPT-1은 대규모 비지도 텍스트로 Transformer 언어모델을 먼저 사전학습(pre-training)한 뒤, 적은 양의 라벨 데이터로 각 NLP task에 fine-tuning하면 다양한 작업에 잘 전이될 수 있다는 것을 보여준 논문

- **Input:** 자연어 문장을 토큰 시퀀스로 변환한 것. 예: ["The", "cat", "sat", ...]
- **Output:**
 - **Pre-training 때:** 각 위치에서 다음 토큰의 확률분포를 예측



- **Fine-tuning** 때: 마지막 hidden representation을 이용해 해당 **task**의 정답을 출력함. 예: NLI의 entailment / contradiction / neutral, 분류 label 등

즉 핵심 흐름은 텍스트 입력 → **Transformer** → **hidden representations** → 다음 단어 예측 또는 **downstream task** 정답 출력

1. Introduction

- 논문의 저자들이 주장하고자 하는 background: Unlabeled dataset ↔ Labeled dataset 을 비교했을 때 Unlabeled dataset 의 양이 훨씬 많음.

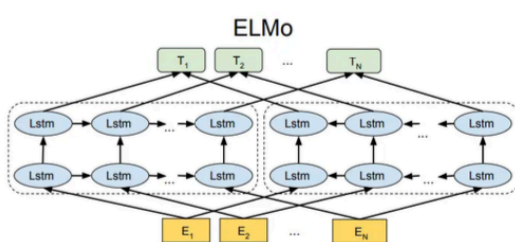
→ Unlabeled dataset 의 양이 훨씬 많으므로, 잘 활용을 하면 더 좋은 performance 가 나오지 않겠느냐?

- Unlabeled Text Corpora → Generative pre-training of a language model 을 통해서 임베딩 벡터를 찾아냄

우리가 하고자 하는 Specific 한 Task 에 대해서 (= label 이 존재하는 task) 조금 더 fine-tuning 을 하게 되는게 더 도움이 되지 않겠냐?

1. Introduction

Background – Prior Approach: ELMo



1. Bidirectional Language Model

Forward + Backward LSTM으로 앞·뒤 문맥을 모두 학습

2. Contextualized Representation

같은 단어라도 문맥에 따라 다른 representation 생성

3. Feature-based Transfer

사전학습된 ELMo representation을 downstream model의 feature로 사용

ELMo: representation을 transfer → GPT: pretrained model 자체를 fine-tuning

? GPT 와 ELMo 의 차이점은 무엇인가?

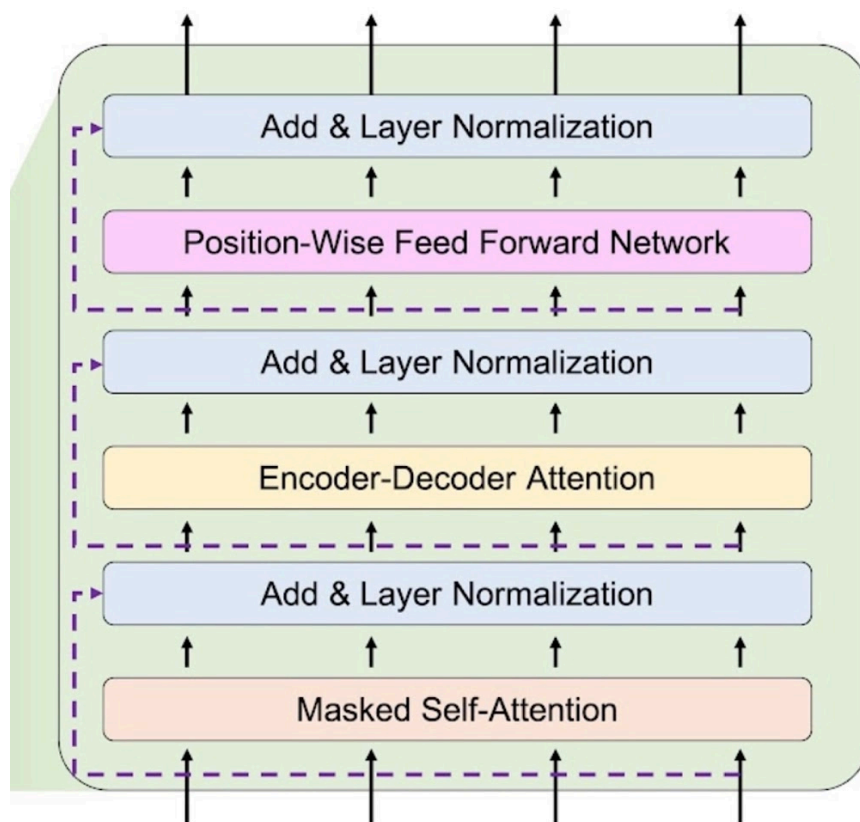
- **ELMo** 는 Bidirectional Language Model 를 사용(Forward LSTM + Backward LSTM)

- $E_1, E_2, \dots, E_N$: 각 토큰의 초기 입력 표현
- 가운데 LSTM 들: 순방향/역방향 LSTM 층
- $T_1, T_2, \dots, T_N$: 각 위치에서 최종적으로 얻는 문맥이 반영된 단어 표현

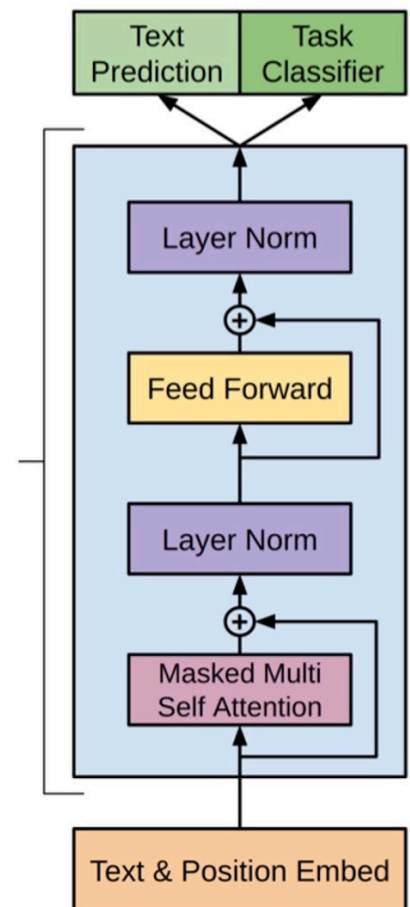
$\mathrm{ELMo}k = \gamma \sum_j s_j h_{k,j}$ “각각의 LSTM layer에서 나오는 hidden node 들의 선형 결합”

- **GPT-1** 는 Transformer 의 Decoder Block 을 사용함.

GPT 는 backward 를 쓰지 않고 forward 에 대해서도 masking 한 것을 사용함.



Transformer Decoder



GPT-1

GPT-1 에서 제시하는 문제점

Remaining Challenges

What to learn? → 어떤 objective가 transfer에 유용한 representation을 가장 잘 학습하는가?
How to transfer? → 학습한 representation을 downstream task에 어떻게 전달할 것인가?

Pre-training은 가능해졌지만, “무엇을 학습하고 어떻게 전이시킬지”는 아직 정립되지 않았다.

라벨이 없는 텍스트에서 단어 수준을 넘어서는 정보를 효과적으로 활용하는 것은 어렵다. → 단순히 unlabeled text 만 가지고서는 어떠한 목적함수가 효과적인지 잘 모르기 때문.

1. 라벨 없는 텍스트를 어떻게 학습시킬 것인가?

2. 그렇게 배운 지식을 다른 작업에 어떻게 가져다 쓸 것인가? → 사전학습으로 얻은 표현을 실제 목표 작업에 어떻게 옮겨 쓰는 것이 가장 좋은지 합의된 방법이 없었다. → 가장 transfer learning 을 하기 위한 가장 effective 한 방식이 무엇인지도 (분류, QA 등) 정해져있지 않음.

3. Framework

3.1 Unsupervised pre-training



Unsupervised Pre-training

Language Model의 사전학습 목적함수

3. Framework

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

이전 token들을 이용해 다음 token 예측
별도의 task label 없이 raw text 자체를 학습 데이터로 사용
전체 sequence의 log-likelihood를 최대화

$U = (u_1, \dots, u_n)$

Unlabeled token sequence

k

사용하는 context 범위

Θ

Model parameters

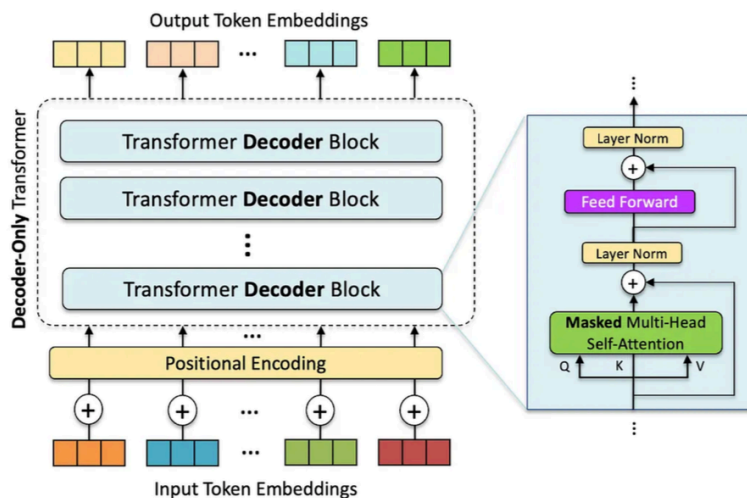
주어진 이전 문맥에서 다음 token을 가장 잘 예측하도록 학습

L_1 목적함수는 $i = 1$ 부터 모든 시퀀스에 대해서 바로 전단계 ~ k 번째 이전까지 살펴본 뒤, 그것을 바탕으로 i 번째에 해당하는 토큰이 무엇인가에 대한 likelihood 를 가장 최대화 시키는 것이 Language Model 의 목적임.

GPT-1의 구조

3. Framework

GPT-1 Architecture



Input Representation

Text Embedding + Position Embedding
Context length: 512

Transformer Decoder × 12

Masked Multi-Head Self-Attention
(Future tokens are masked)

- Position-wise Feed-Forward
- 768 hidden / 12 heads / FFN 3072

Output

Pre-training → Text Prediction
Fine-tuning → Task Classifier

GPT 는 multi-layer Transformer decoder 구조.

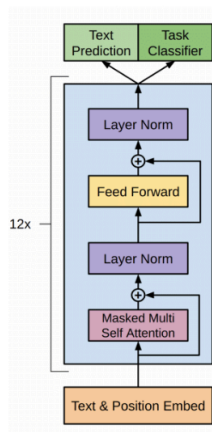
masked multi-attention 만을 사용하고, 여러 층으로 쌓음.

Transformer 의 Decoder 블록만을 쌓는게 GPT 의 아이디어.

- Multi-Head: 한 Transformer 블록 안에서 attention head를 여러 개 사용
- Multi-Layer: Transformer 블록 자체를 여러 층 쌓음

Unsupervised Pre-training

Language Model의 계산 과정



1. 입력 표현 만들기

$$h_0 = UW_e + W_p$$

Token Embedding과 Position Embedding을 더해 Transformer의 입력 표현을 생성

U : input token sequence

W_e : token embedding matrix

W_p : positional embedding

2. Transformer Decoder Blocks 통과

$$h_l = \text{transformer_block}(h_{l-1}) \forall l \in [1, n]$$

각 Transformer block은 이전 block의 출력 표현을 입력으로 받아 새로운 표현을 생성 (여기서 n은 Transformer layer 수)

3. 다음 토큰 확률 계산

$$P(u) = \text{softmax}(h_n W_e^T)$$

마지막 Transformer 출력에서 vocabulary 전체의 다음 token 확률분포를 계산

1. 입력 표현 만들기

$$h_0 = UW_e + W_p$$

- $U = (u_{-k}, \dots, u_{-1})$: 현재 토큰을 예측할 때 보는 이전 토큰들의 시퀀스
- W_e : 토큰 임베딩 행렬
- W_p : 위치 임베딩 행렬

h_0 는 **Transformer**에 들어가기 직전의 입력 표현 전체

2. Transformer 블록 여러 층 통과

$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall l \in [1, n]$$

- l : 현재 층 번호
- h_{l-1} : 이전 Transformer 층의 출력



- h_l : 현재 Transformer 층의 출력
- n : 전체 Transformer 층 개수
- $\forall$: 모든
- $\in$: ~에 속함
- $[1, n]$: 1층부터 n 층까지

1층부터 $n-1$ 층까지 모든 Transformer 블록에 대해, 이전 층의 출력을 현재 층에 입력한다.

3. 토큰 확률 계산

$$P(u) = \text{softmax}(h_n W_e^T)$$

- h_n : 마지막 Transformer 층의 출력
- W_e^T : 입력에서 사용했던 임베딩 행렬 W_e 의 전치행렬
- $P(u)$: 어휘 전체에 대한 다음 토큰 확률
- **? Multi-Head vs Multi-layer**

Multi-Head

= 한 Transformer 블록 안에서 attention head를 여러 개 사용

Multi-Layer

= Transformer 블록 자체를 여러 층 쌓음

3.2 Supervised fine-tuning

입력 토큰들 → 사전학습된 GPT → 마지막 토큰의 표현 → Linear → Softmax → 정답 y 예측

label y 를 가지고 있는 토큰들의 시퀀스 $(x^1, x^2, \dots, x^m, y)$ 가 들어오게 되면, 이것 통해서 input 들은 pre-trained

y 는 해당 입력의 정답 label

C 는 이런 데이터들을 모아놓은 labeled dataset

$$P(y \mid x^1, \dots, x^m) = \text{softmax}(h_l^m W_y)$$



- $x^1, \dots, x^m$: 입력 토큰 시퀀스
- m : 입력 시퀀스의 마지막 토큰 위치
- l : 마지막 Transformer 층
- h_l^m : 마지막 Transformer 층에서, 마지막 토큰 위치의 은닉 표현 (final TRM block's activation)
- W_y : 새로 붙인 분류용 선형층의 가중치
- y : 정답 라벨
- $P(y \mid x^1, \dots, x^m)$: 입력이 주어졌을 때 각 라벨일 확률
- softmax: 점수를 확률분포로 변환

$$L_2(C) = \sum_{(x,y)} \log P(y \mid x^1, \dots, x^m)$$

- C : 라벨이 붙어 있는 전체 미세조정 데이터셋
- (x, y) : 하나의 학습 데이터
 - x : 입력
 - y : 정답 라벨
- $\sum (x, y)$: 모든 학습 데이터에 대해 더함
- $P(y \mid x^1, \dots, x^m)$: 정답 라벨에 모델이 부여한 확률
- $L_2(C)$: 지도 미세조정의 목적함수

$$L_3(C) = L_2(C) + \lambda L_1(C)$$

- $L_3(C)$: GPT-1 미세조정에서 실제 사용하는 최종 목적함수
- $L_2(C)$: 정답 라벨을 맞추는 지도학습 목적
- $L_1(C)$: 다음 토큰을 맞추는 기존 언어모델 목적
- λ : 두 번째 목적을 얼마나 강하게 반영할지 조절하는 가중치

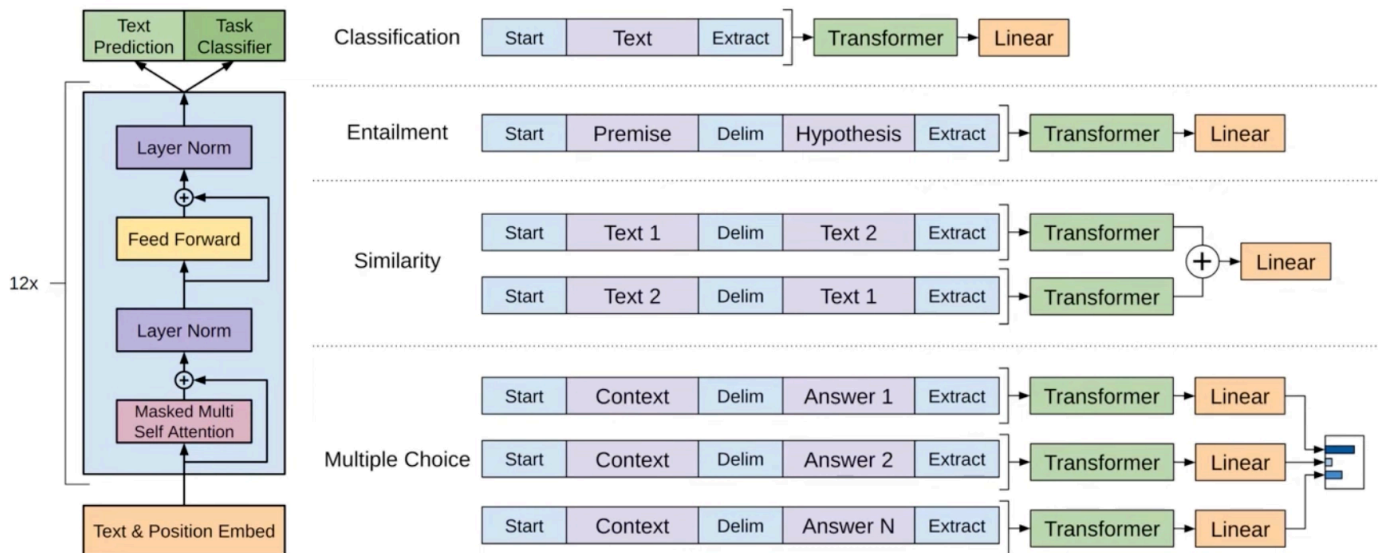
($L_1(U)$: unsupervised 전체 task corpus 에 대한 language model 의 목적함수 로 pre-training 을 하고, C 는 supervised learning task 에 대한 corpus 임.)

★ 저자들이 추가적으로 알아낸 바는 L_2 뿐만이 아니라 L_1 도 함께 C 로 업데이트 하게 되면,



1. supervised model 에 대한 generalization ability 가 더 향상됨.
2. convergence(학습속도)가 더 빨라짐.

이라는 2가지의 장점이 있었음.



3.3 Task-specific input transformers

(초록색 Transformer = GPT-1의 Transformer 블록 12층 전체를 통과한 것. Linear나 Softmax까지 포함한 게 아님. 사실상, 마지막 hidden representations 임.)

- **Entailment** 같은 경우에는, Premise 에 해당하는 문장, Hypothesis 에 해당하는 문장을 delimiter 로 구분하여 집어넣고, output transformer block 결과물을 이용해서 linear layer 를 태움.

- **Entailment = 함의**, 자연어 추론 문제

두 문장이 있을 때 첫 번째 문장이 참이라면 두 번째 문장도 참이라고 볼 수 있는가?를 판단함.

Premise (전제) :

"민준이는 서울에서 공부하고 있다."

Hypothesis (가설) :

"민준이는 한국에서 공부하고 있다."

→ Entailment

Similarity 같은 경우에는, text 가 2개 들어가는데 두 문장을 순서를 바꾼 뒤 transformer block 에 집어넣은 뒤, output 2개를 concat 을 한 뒤 linear layer 에 입력으로 집어넣음.

Multiple Choice 같은 경우에는, context 와 각각의 answer 에 대한 조합을 넣고 TRM 의 decoder 에 태우고, 나오는 output hidden state vector 각각을 구한 뒤 전부 linear layer 에 투입을 하고 거기에서 나온 결과물을 통해 softmax 를 계산함.

4. Experiments

논문의 핵심 주장은 대략:

대규모 language-model pre-training → downstream supervised fine-tuning

을 하면 하나의 Transformer architecture가 다양한 NLU task에 transfer될 수 있다는 것임. 논문은 NLI, QA, semantic similarity, classification이라는 네 종류의 task에서 이를 시험함.

4.1 Setup (“실험을 어떤 조건에서 했는데?”)

Unsupervised pre-training (첫 단계가 pre-training 이었으므로 어떤 데이터로 pre-training했는가?)

pre-training dataset: BooksCorpus dataset (문장이 서로 연결된 긴 contiguous text가 존재하기 때문)

token-level perplexity 는 언어모델이 다음 토큰을 얼마나 헛갈려 하는지를 나타내는 지표임.

GPT-1 논문에서는 BooksCorpus에서 pre-training한 언어모델이 token-level perplexity 18.4를 얻었다고 씀.

Model Specifications (“그 데이터를 무슨 모델로 학습했는데?”)

12L / 768H / 12 heads

Fine-tuning details (“Fine-tuning은 어떤 설정으로 했는데?”)

항목	GPT-1 설정	의미
Learning rate	6.25×10^{-5}	한 번 업데이트할 때 가중치를 얼마나 움직일지
Batch size	32	한 번의 업데이트에 사용할 학습 데이터 개수
Epochs	약 3	전체 학습 데이터를 몇 바퀴 학습하는지
Classifier dropout	0.1	classifier 쪽 뉴런 출력을 학습 중 10% 무작위로 끄기
λ	0.5	auxiliary LM objective를 얼마나 강하게 반영할지

GPT-1 downstream 구조를 단순화하면:

```
Input
↓
Pre-trained Transformer
↓
Hidden Representation
↓
Dropout
↓
Linear Classifier
↓
Softmax
↓
entailment / neutral / contradiction
```

여기서 `dropout = 0.1` 이면 학습 중 해당 representation의 일부를 10% 확률로 무작위로 꺼버림.

dropout 0.1 = 학습할 때 일부 정보를 10% 확률로 끄면서 더 robust하게 학습

(robust하게는 여기서는 그냥 “조금 조건이 바뀌거나 일부 정보가 없어도 성능이 쉽게 무너지지 않게”라는 뜻임.)

Auxiliary LM objective는 말 그대로 보조(auxiliary) 언어모델링 목표(objective)임.

Auxiliary LM objective = downstream 문제를 배우면서, 원래 GPT의 “다음 단어 맞히기”도 보조 과제로 같이 시키는 것.

4.2 Supervised fine-tuning

위에서 이제 실험 환경을 다 설명했음.

“그래서 실제 downstream task에서 성능이 어땠는데?”

가 나와야 함.

- ? NLI 가 왜 먼저 등장했는가?

하나의 모델이 다양한 language understanding task에 transfer 가능하다.

그래서 한 종류만 실험하면 안 됨.

NLI: 문장 관계를 이해할 수 있나?



QA / Commonsense reasoning: 긴 글을 읽고 추론할 수 있나?

Semantic Similarity: 두 문장의 의미가 같은지 판단할 수 있나?

Classification: 문법성이나 감정 같은 속성도 판단할 수 있나?

5. Analysis

Impact of number of layers transferred

“Pre-training된 깊은 layer들이 진짜 도움이 되는가?”

→ 가져오는 layer 수를 늘려봄 → 성능도 대체로 증가

따라서 pretrained layer들이 유용하다는 근거

Zero-shot Behaviors

“Fine-tuning 전에 이미 뭔가 언어 능력을 배우고 있었던 건 아닐까?”

→ pre-training 도중 sentiment, QA, grammaticality 등을 직접 검사

→ 학습이 진행될수록 능력 증가.

Ablation studies

“우리가 넣은 각 요소가 진짜 필요한가?”

pre-training 제거 → 크게 하락

Transformer → LSTM → 하락

auxiliary LM 제거 → 효과가 task마다 다름

Further Reading

Sep 10, 2026

[Paper] LoRA: Low-Rank Adaptation of Large Language Models

Introduction LoRA는 거대 인공지능 모델의 전체 가중치를 수정하는 대신, 적은 수의 추가 파라미터만 학습시켜 비용과 시간을 크게 줄이는 고효율 파인튜닝 기법이다. Model Size가 커졌고, 대형언어모델은 GPU에 올리는 것조차 쉽지 않음...

OLDER	NEWER
[Paper] LoRA: Low-Rank Adaptation of Large Language Models	-

